The Frequency Domain

Week 6 — Session 1: Seeing images as sums of waves

Practical Image Processing

Lecture 6.1 ▪ 60–70 min

Recap & today

- Wk3–5: filters in the **spatial** domain (pixels & kernels).
- Today (Session 1): a whole new coordinate system for images — the **frequency** domain. Every image is a sum of waves.
- We build the intuition, learn to *read* a spectrum, and do low/high-pass filtering by masking.

Why it matters

A striped/patterned scan of a meter is nearly impossible to clean spatially — but it's a few dots in the frequency domain. (We tackle that in Session 2.)

Roadmap

The Fourier idea

Reading the spectrum

Filtering in frequency

In-class exercise

Wrap-up

The Fourier idea

Any signal = a sum of sine waves

- **Fourier**: any image can be written as a sum of 2D sinusoids of different **frequency**, **orientation**, and **amplitude**.
- Low frequencies = smooth, slow variation (overall brightness, big shapes).
- High frequencies = fast variation (edges, fine texture, noise).

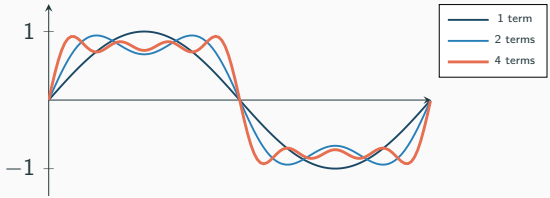
Math

2D DFT: $F(u,v) = \sum_{x=0}^{W-1} \sum_{y=0}^{H-1} f(x,y)\, e^{-j2\pi(ux/W + vy/H)}$

Key idea

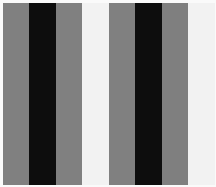
The transform doesn't change the image — it re-describes it. Same information, different axes: instead of "brightness at each pixel" we store "how much of each wave."

1D intuition: building a square wave from sines



- Add odd harmonics $\sin x + \frac{1}{3}\sin 3x + \frac{1}{5}\sin 5x + \ldots$
- More terms $\rightarrow$ sharper corners.
- **Edges need high frequencies**; that is why blurring (dropping high freq) rounds corners.

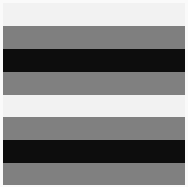
2D sinusoids: frequency, orientation, amplitude



horizontal, low freq



horizontal, high freq



vertical orientation

A single 2D sinusoid is a set of parallel stripes. Three numbers describe it: **frequency** (stripe spacing), **orientation** (stripe direction), **amplitude** (contrast). The whole image is a weighted sum of these.

Worked example: a tiny 1D DFT by hand

Math

Signal $f = [4,\ 0,\ 4,\ 0]$ (a 2-sample-period stripe). $N = 4$. $\quad F(u) = \sum_{x=0}^{3} f(x)\, e^{-j2\pi ux/4}$

- $F(0) = 4 + 0 + 4 + 0 = 8$ (the **DC** term = sum = $4\times$ the average).
- $F(1) = 4\, e^{0} + 0 + 4\, e^{-j\pi} + 0 = 4 - 4 = 0.$
- $F(2) = 4\, e^{0} + 0 + 4\, e^{-j2\pi} + 0 = 4 + 4 = 8$ (energy at the **highest** freq).
- $F(3) = 0$ (mirror of $F(1)$).

Key idea

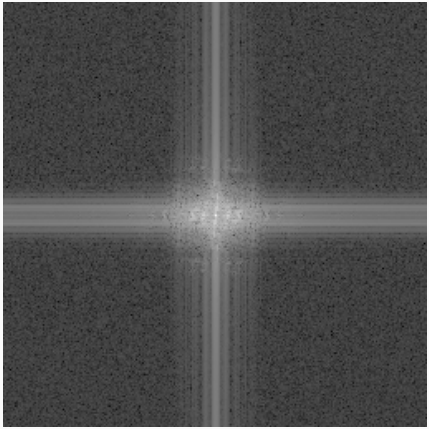
$[4, 0, 4, 0]$ alternates every sample $\rightarrow$ all its non-DC energy sits at the maximum frequency $u = 2$. The numbers match the picture: a fast stripe.

Reading the spectrum

The spectrum of a real plate



image (spatial)



magnitude spectrum (log)

- Centre = low frequencies (DC = average brightness). Edges = high frequencies.
- Bright lines/points reveal dominant orientations and repeating structure.

Reading the spectrum: three rules

- The **centre** is the average; energy usually concentrates there (natural images are mostly low-freq).
- A strong **direction** in the image $\rightarrow$ a bright line *perpendicular* to it in the spectrum.
- **Repeating patterns** (stripes, halftone) $\rightarrow$ isolated bright **peaks** off-centre.

Note

We take the `log` of the magnitude to see anything — the DC term dwarfs everything else.

Why we take the log of the magnitude

Raw magnitudes span a huge range. Example values along one row:

60000	900	300	90	40

On a linear scale only the DC pixel is visible; the rest look black.

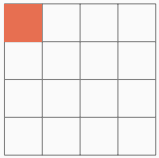
After $20 \log_{10}(1 + |F|)$:

95	59	50	39	32

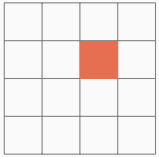
✔ Good practice

The $+1$ inside the log avoids $\log 0$. The log compresses 4 orders of magnitude into a readable image.

Centering the DC: `fftshift`



raw: DC at corner



after fftshift: DC centred

- Raw FFT output puts the zero frequency (DC) at the top-left corner.
- `fftshift` swaps quadrants so DC lands in the middle — the way we always draw and reason about spectra.
- Remember to `ifftshift` back before the inverse transform.

A gallery of simple spectra



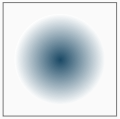
flat image

one central dot



vertical stripes

DC + symmetric pair



smooth photo

energy hugging centre

Learn these three archetypes and you can read most spectra by inspection.

Symmetry: a real image's spectrum is a mirror

Math

For a real-valued image, $F(-u,-v) = \overline{F(u,v)}$ (conjugate symmetry). So $|F|$ is symmetric through the centre: $|F(-u,-v)| = |F(u,v)|$.

- Every bright point has an equally bright **twin** on the opposite side of the DC.
- That is why stripes always show *two* peaks, and why we notch them in pairs (Session 2).
- Half the spectrum is redundant — real-FFT routines exploit this to save memory.

Filtering in frequency

Low-pass & high-pass, done in frequency



original



low-pass = blur



high-pass = edges

Keep the centre (low freq) $\rightarrow$ smoothing. Remove the centre (keep high) $\rightarrow$ edges. Same effects as Week 3, achieved by *masking* the spectrum.

Convolution theorem: why this matters

Math

Convolution in space = multiplication in frequency: $f * h \longleftrightarrow F \cdot H$

- A big spatial convolution becomes a cheap element-wise multiply in frequency.
- For large kernels, FFT-based filtering is much faster ($O(N \log N)$ vs $O(Nk^2)$).
- It's also *why* the spatial and frequency views are two sides of one coin.

Worked example: when does the FFT pay off?

Math

Image $N = 1024^2 \approx 10^6$ pixels. Spatial cost $\approx Nk^2$; FFT cost $\approx N \log_2 N$.

- $\log_2 N \approx 20$, so FFT filtering costs $\approx 2 \times 10^7$ ops (independent of kernel size).
- A 3×3 kernel: spatial $\approx 9 \times 10^6$ ops — **spatial wins** for tiny kernels.
- A 31×31 kernel: spatial $\approx 10^9$ ops — **FFT wins** by $\sim 50\times$.

Key idea

Break-even is around a 15–20 pixel kernel. Small blur: stay spatial. Large blur or a full-image mask: go to frequency.

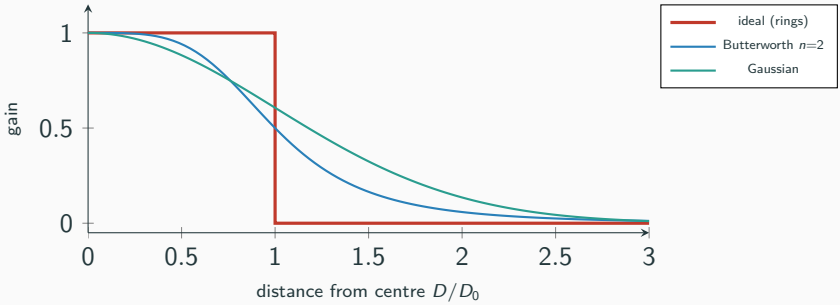
Ideal vs smooth filters (ringing)

- An **ideal** sharp cutoff (a hard circle) causes **ringing** — ripples near edges.
- Smooth cutoffs (**Butterworth**, **Gaussian**) avoid ringing — the usual choice.
- Trade-off: sharper frequency selectivity $\leftrightarrow$ more spatial ringing.

✕ Pitfall

A hard frequency mask can add wavy artifacts to a clean image. Prefer a gradual roll-off.

The shape of the roll-off



The ideal filter's vertical wall is what injects ringing; the smooth curves taper the transition and stay artifact-free.

Fourier in code

```
F = np.fft.fftshift(np.fft.fft2(gray))      # to frequency, centre the DC
mag = 20*np.log(np.abs(F) + 1)              # log magnitude for display
# low-pass: keep a central disc
mask = make_circle_mask(gray.shape, r=30)
out = np.abs(np.fft.ifft2(np.fft.ifftshift(F * mask)))  # back to spatial
```

`fft2` → mask → `ifft2`. The whole frequency workflow in three steps.

Separability: 2D FFT = rows then columns

Math

$$F(u,v) = \sum_{x} \Bigg[\underbrace{\sum_{y} f(x,y)\, e^{-j2\pi vy/H}}_{\text{1D FFT of each column}} \Bigg] e^{-j2\pi ux/W}$$

- The 2D transform **separates**: FFT every column, then FFT every row of the result.
- This is why the 2D FFT is only $O(N \log N)$ overall, not something worse.
- `np.fft.fft2` does both passes for you — but knowing it is two 1D sweeps demystifies it.

Note

The same separability is why a 2D Gaussian blur can be done as a horizontal then a vertical 1D blur (Week 3) — the two domains echo each other.

Building the circular mask

```python
def make_circle_mask(shape, r):
    H, W = shape
    cy, cx = H // 2, W // 2                 # centre = DC after fftshift
    Y, X = np.ogrid[:H, :W]
    D = np.sqrt((X - cx)**2 + (Y - cy)**2)  # distance from centre
    return (D <= r).astype(float)           # 1 inside the disc, 0 outside
# high-pass is just the complement:
hp = 1.0 - make_circle_mask(gray.shape, r=30)
```

Note

Swap `D <= r` for a Gaussian `np.exp(-(D**2)/(2*r**2))` to get a smooth, ring-free roll-off.

In-class exercise

In-class exercise (10 min)

Predict the spectrum

For each image, sketch what its magnitude spectrum looks like:

1. A perfectly flat gray image.
2. Vertical black-and-white stripes.
3. A photo with lots of fine texture vs a smooth gradient.
4. Where in the spectrum would you look to detect a repeating scan pattern?
5. Bonus: for a 31×31 blur on a 1024^2 image, is FFT or spatial faster?

Debrief

- (1) Flat $\rightarrow$ a single point at the centre (only DC; no variation).
- (2) Vertical stripes $\rightarrow$ two symmetric points on the horizontal axis (one frequency).
- (3) Fine texture $\rightarrow$ lots of energy far from centre; smooth $\rightarrow$ energy hugging the centre.
- (4) Look for isolated bright peaks *off-centre* — that's periodic structure.
- (5) FFT: $31^2 \approx 960$ ops/pixel spatially vs ~ 20 for FFT $\Rightarrow$ FFT wins.

Key idea

The spectrum is a map of "how much of each wave." Reading it becomes intuition with practice.

Wrap-up

Pitfalls & good practice so far

✗ Pitfall

- Forgetting `fftshift`/`ifftshift` → a scrambled result.
- Displaying $|F|$ without a log → an all-black image.
- A hard mask → ringing.

✓ Good practice

- Always centre the DC before reasoning.
- Use a smooth roll-off (Gaussian/Butterworth).
- Take `np.abs` of the inverse — round-off leaves a tiny imaginary part.

What you should now be able to do

- Explain an image as a sum of waves; hand-compute a tiny DFT.
- Read a magnitude spectrum: centre, directions, off-centre peaks.
- Low/high-pass filter by masking; choose ideal vs smooth and know why.
- Use the convolution theorem to decide spatial vs FFT.

Next session

Session 2 — the killer application

- **Periodic noise**: repeating stripes are two dots in frequency — zap them with a **notch** filter.
- **Phase vs amplitude**, the FFT's speed, and the transform family (DCT, wavelets).
- **Aliasing** and **homomorphic** lighting correction, in the frequency view.
- Then the week's deliverable.

An image is a sum of waves.

Change the axes, and hard problems get simple. Next: making that pay off.